OR114-2 Midterm Project

Group 4

B13705051 周孟承 B13902103 鄭宇宏 B13303153 詹舒宇 B11705039 李盈盈

Problem 1

1. Mathematical Formulation

Let $S = \{1, ..., n_S\}$ be the set of stations, $C = \{1, ..., n_C\}$ be the set of cars, $L = \{1, ..., n_L\}$ be the set of levels, and K be the set of orders. Each car $c \in C$ has initial station $h_c \in S$ and level $q_c \in L$. Each order $k \in K$ has requested level $\ell_k \in L$, pickup & return stations $p_k, r_k \in S$, pickup time s_k, return time e_k, and revenue R_k. Let $t_{u,v}$ be the moving time from station u to station v, both $u, v \in S$; and B be the total relocation-time budget.

We build a directed acyclic route network. Let $\tau(t)$ be the function that converts time t to minutes from the start of the planning horizon (00:00). The set of feasible initial arcs A^0 and transition arcs A are strictly defined by level compatibility and time constraints described below:

$$A^0 = \{(c, k) \in C \times K \mid q_c \in \{\ell_k, \ell_k + 1\} \text{ and } t_{h_c,p_k} \leq \tau(s_k) - 30\}$$

$$A = \left\{(c, i, j) \in C \times K \times K \,\middle|\, q_c \in \big(\{\ell_i, \ell_i + 1\} \cap \{\ell_j, \ell_j + 1\}\big) \text{ and } \tau(e_i) + 240 + t_{r_i,p_j} \leq \tau(s_j) - 30\right\}$$

Let $m_{c,k} = t_{h_c,p_k}$ for $(c, k) \in A^0$, and let $m_{i,j} = t_{r_i,p_j}$ for $(c, i, j) \in A$.

With the variables defined above, the IEDO planning problem can be formulated as the following integer program.

$$\begin{array}{lll}
\max & \displaystyle\sum_{k\in K} 3R_k y_k & \\
\text{s.t.} & \displaystyle\sum_{c:(c,k)\in A^0} x_{ck} + \sum_{c\in C} \sum_{i\in K:(c,i,k)\in A} z_{cik} = y_k & \forall k \in K, \\
& \displaystyle\sum_{k:(c,k)\in A^0} x_{ck} \leq 1 & \forall c \in C, \\
& \displaystyle\sum_{j:(c,i,j)\in A} z_{cij} \leq \sum_{k\in\{i\}:(c,k)\in A^0} x_{ck} + \sum_{h:(c,h,i)\in A} z_{chi} & \forall c \in C,\ i \in K_c, \\
& \displaystyle\sum_{(c,k)\in A^0} m_{c,k} x_{ck} + \sum_{(c,i,j)\in A} m_{i,j} z_{cij} \leq B, & \\
& x_{ck} \in \{0, 1\} & \forall (c, k) \in A^0, \\
& z_{cij} \in \{0, 1\} & \forall (c, i, j) \in A, \\
& y_k \in \{0, 1\} & \forall k \in K.
\end{array}$$

2. Optimal Solution for Instance 05

The optimization model accepts 8 out of 10 orders, including Orders 3, 4, 5, 6, 7, 8, 9, and 10. Orders 1 and 2 are rejected. The 8 completed orders generate an accepted sales revenue of 117,000. The rejected orders require compensation of $2 \times 1{,}900 + 2 \times 3{,}200 = 10{,}200$, so the final profit after compensation is $117{,}000 - 10{,}200 = 106{,}800$. The total relocation time used is 1,080 minutes, which satisfies the relocation budget limit ($1080 \leq 1200$). Only one vehicle upgrade is required in the final solution, where a level-2 vehicle is assigned to Order 6, which requests a level-1 vehicle.

3. Operational and Relocation Plans

The relocation instructions needed before serving an order are integrated in Table 1, so an operator can execute the accepted plan without cross-checking another schedule. The detailed vehicle relocation schedule is summarized in Table 2.

Table 1: Operational Plan for Instance 05

Order	Car	Upgrade	Pre-Move (From → To)	Move Mins	Pickup → Return	Rental Period	Revenue
5	6	No	-	0	6 → 6	01/01 00:00 – 01/04 20:00	36,800
10	2	No	2 → 4	240	4 → 2	01/01 06:00 – 01/05 06:00	9,600
9	4	No	5 → 3	210	3 → 4	01/01 19:30 – 01/04 15:30	27,200
3	3	No	3 → 5	210	5 → 1	01/02 03:30 – 01/04 11:30	5,600
7	5	No	4 → 2	240	2 → 1	01/02 03:30 – 01/04 23:30	27,200
4	1	No	-	0	1 → 2	01/03 12:00 – 01/05 15:00	5,100
6	4	Yes	4 → 3	180	3 → 4	01/04 23:00 – 01/05 14:00	1,500
8	6	No	-	0	6 → 6	01/05 04:30 – 01/05 14:30	4,000

Table 2: Vehicle Relocation Plan for Instance 05

Car	From	To	Depart	Arrive	Minutes
2	2	4	01/01 00:00	01/01 04:00	240
3	3	5	01/01 00:00	01/01 03:30	210
4	5	3	01/01 00:00	01/01 03:30	210
4	4	3	01/04 19:30	01/04 22:30	180
5	4	2	01/01 00:00	01/01 04:00	240

Problem 3

The hidden instances are graded under a three-minute time limit. This constraint is the main reason for our design. A full IP can make globally good decisions, but it is too expensive on large instances. A purely greedy method is fast, but it can waste scarce relocation minutes early. We therefore divide the algorithm into two parts:

- **Pre-build:** construct feasible solutions very quickly using two different approaches proposed by our team.
- **Optimization:** spend the remaining time on batch local IP repair, improving only small neighborhoods of the current solution.

In implementation, the pre-build stage is intended to finish quickly and every local IP call in optimization stage receives only a short time limit. We always check the wall-clock deadline before starting another repair.

Pre-build Approach A: Revenue-first route insertion

Approach A is designed to be a fast, reliable high-profit route generator using a single-pass greedy insertion process:

- **Order prioritization.** Orders are sorted primarily by decreasing revenue and secondarily by pickup time. The purpose is to give high-value orders early access to cars and relocation budget.
- **Chronological insertion search.** Each car maintains a chronological route. For an order k, the method scans compatible cars, i.e., exact level matches and valid one-level upgrades, and uses the pickup time to locate where k would be inserted in the route.
- **Feasibility evaluation.** The transition from the predecessor order to k and the transition from k to the successor order are checked. These tests enforce the 1-hour possible late return, 3-hour cleaning, thirty-minute readiness requirement, station moving time, level compatibility, and remaining global relocation budget.
- **Lexicographical selection.** Among all feasible insertions for the current order, the algorithm chooses the minimum lexicographical key: marginal moving time → upgrade penalty → local idle time → car ID.

If i and j are the predecessor and successor around k, the marginal moving time is

$$\Delta\text{move} = T_{r_i,p_k} + T_{r_k,p_j} - T_{r_i,p_j},$$

with the obvious modification when k is inserted as the first or last order of a car route. The other three terms in the selection rule are defined exactly as in the implementation. The upgrade penalty is $q_c - \ell_k$ which is 0 for an exact-level assignment and 1 when a one-level upgrade is used. The local idle time is the idle slack around the inserted order:

$$I_{\text{local}} = \mathbf{1}_{i\text{ exists}}\big(s_k - \text{ready}(i)\big) + \mathbf{1}_{j\text{ exists}}\big(s_j - \text{ready}(k)\big),$$

where ready(i) is the earliest time after order i can be followed by another service, including the allowed late return, cleaning time, and readiness buffer used in the feasibility check. The car ID is used only as a deterministic tie-breaker. Therefore, the exact key minimized by the selection rule is $(\Delta\text{move},\ q_c - \ell_k,\ I_{\text{local}},\ c)$.

Thus Approach A prioritizes revenue while still conserving relocation budget and avoiding unnecessary upgrades. For very small instances, an exact solver may be used as a fallback, but this step is bypassed for large hidden instances. Let $\bar{r}$ be the average route length. Sorting orders costs $O(n_K \log n_K)$. For each order, the algorithm may check n_C cars and inspect a local insertion position. Since $\bar{r}$ is at most the number of orders already processed, $\bar{r} \le n_K$, the construction cost is $O(n_K \log n_K + n_K n_C \bar{r}) \subseteq O(n_K^2)$. This is a deliberately loose upper bound: in practice each route is much shorter than n_K, only one chronological insertion neighborhood is checked, and the method runs very quickly.

Pre-build Approach B: Relocation-aware look-ahead heuristic

The second pre-build approach is a relocation-aware multi-variant heuristic. Its main purpose is to construct a strong feasible solution quickly while treating employee moving time as a scarce global resource. Besides looking ahead to future spatial demand, it also contains a staging step and a relocation-value variant, so it can perform better when the initial car distribution and early customer demand are spatially mismatched.

1. Demand-aware Look-ahead Scoring

The planning horizon is divided into 6-hour buckets. For each station, car level, and bucket, we estimate future demand value. The compatibility check is written from the order side: an order requesting level r can be served by a car of level r or $r+1$. Therefore each future pickup of requested level r adds its full value to the table entry for level r at its pickup station, and also adds a discounted value, $0.75 \times 3R_k$, to the entry for level $r+1$, because a level-$r+1$ car could serve this order by upgrade. A rolling 4-bucket window, i.e., about 24 hours, is then used to estimate how useful a car will be near a station in the near future.

When evaluating whether car c should serve order k, the method uses

$$\text{Score} = 3R_k + w_f F(s_{\text{ret}}, \ell_c, b_{\text{ret}}) - w_o F(s_{\text{cur}}, \ell_c, b_{\text{cur}}) - P_{\text{move}} - w_i I - w_u \cdot U \cdot \max(1, \ell_c).$$

Here s_{cur} is the car's current station before serving the order, s_{ret} is the order's return station, and ℓ_c is the car level. And b_{cur} and b_{ret} are 6-hour bucket time indices computed from the car's current available time and the ready time after serving the order. The future-demand table is computed directly from the remaining order list:

$$D_{s,\ell,b} = \sum_{\substack{k: p_k = s,\ \ell_k = \ell \\ \lfloor t_k^p / 360 \rfloor = b}} 3R_k + \sum_{\substack{k: p_k = s,\ \ell_k + 1 = \ell \\ \lfloor t_k^p / 360 \rfloor = b}} 0.75 \cdot 3R_k, \quad F(s, \ell, b) = \sum_{\tau = b}^{b+3} D_{s,\ell,\tau}.$$

Thus $F(s, \ell, b)$ is the next-24-hour value of placing a level-ℓ car at station s at bucket b. The term I is the idle time before the car can serve the order, and U is an upgrade indicator: $U = 1$ if a level-ℓ_c car serves a level-$(\ell_c - 1)$ request, and $U = 0$ for an exact level match. The moving penalty P_{move} is dynamic:

$$P_{\text{move}} = w_m M \left(1 + \frac{3B_{\text{used}}}{B}\right),$$

where M is the relocation time, w_m is the base moving-time penalty weight, and B_{used} is the relocation budget already consumed. Thus moving becomes increasingly expensive as the remaining budget becomes scarce. The upgrade penalty is also level-aware through $w_u \cdot U \cdot \max(1, \ell_c)$, which discourages wasting higher-level cars on lower-level requests unless the accepted reward and future positioning justify it.

The weights are manually tuned heuristic hyperparameters. w_m, w_i, w_u, w_f, and w_o control relocation-minute penalty, idle-minute penalty, one-level upgrade penalty, future-positioning reward, and current-position opportunity cost, respectively. Moreover, the factor $3R_k$ appears because total profit can be rewritten as

$$\sum_{k \in A} R_k - 2 \sum_{k \notin A} R_k = 3 \sum_{k \in A} R_k - 2 \sum_k R_k$$

so accepting one more order improves the objective by $3R_k$ relative to rejecting it.

At last, for the five fixed score-weight hyperparameters, we run several complementary variants and keep the feasible plan with the largest accepted reward:

variant	w_m	w_i	w_u	w_f	w_o	extra feature
bucket	0.7	0.003	250	0.012	0.006	initial staging
relocation value	0.6	0.002	350	0.012	0.008	$\frac{3R_k}{\text{move}}$ gate
revenue	0.8	0.002	500	0.010	0.010	
time	1.2	0.002	400	0.010	0.008	
density (revenue/min)	0.8	0.004	300	0.015	0.006	

For large instances, only the first two variants are used. They are complementary: one relocates a limited number of idle cars before service, while the other ranks relocations by marginal value per moving minute.

2. The 7-Phase Execution Pipeline

For each car, the implementation stores its current station, next available time, route history, relocation records, and initial staging move. The global relocation budget is updated after every staging move or accepted order. All repair phases replay a proposed single-car route from the car's original station and time zero, accepting it only if level, timing, and budget feasibility hold.

- **Phase 1: Optional Initial Staging.** The staging variant uses the first 24 hours of demand to compute shortage values for each pickup-station and requested-level pair. It moves a capped number of idle compatible cars at time 0 toward high-shortage buckets, subject to the global budget and a staging cap. The score rewards shortage reduction and penalizes long moves, removal from high-future-demand origins, and upgrades. Given $N_{s,\ell}$ as the number of cars of level ℓ initially located at station s and $V_{\text{supply}} = 1200$ as the approximate early $3R$ demand covered by one initially local compatible car, shortage value $SV_{s,\ell}$ is defined as:

$$SV_{s,\ell} = \max\left(0, F(s,\ell,0) - V_{\text{supply}} \cdot N_{s,\ell}\right)$$

- **Phase 2: Initial Greedy Assignment.** After staging, the method scans orders according to the current variant. The available sorting modes include bucketed density, relocation value, revenue-first, chronological, and revenue density. The relocation-value mode sorts orders by six-hour bucket and then by $3R_k/(1 + \min_{c \text{ compatible}} T_{s_c,p_k})$, so orders with high profit improvement per required moving minute receive early access to the relocation budget. If the relocation-value gate is enabled, a move is allowed only when:

$$\frac{3R_k}{M} \geq \lambda \left(1 + \alpha \frac{B_{\text{used}}}{B}\right),$$

  where M is the proposed relocation time, $\lambda = 2$ be the minimum profit-per-moving-minute required when the budget is completely full ($B_{\text{used}} = 0$), ensuring that every move justifies its minimum cost, and $\alpha = 2.5$ as a budget-pressure multiplier. Thus the algorithm becomes more selective as relocation budget is consumed. For each order, we scan only compatible cars, compute the demand-aware score, and choose feasible car with the largest tuple (score, $-$move, $-$upgrade, $-$idle). This phase builds a complete feasible baseline route set.
- **Phase 3: Shortage-bucket Repair.** After Phase 2, the algorithm rebuilds the future-demand table using only rejected orders. It groups these orders by pickup station, six-hour bucket, and requested level. Each group receives a priority weighted by revenue and proximity, so high-value shortage clusters are repaired early, while remote clusters are delayed. Inside each bucket, orders are tried by decreasing revenue and assigned with the same scoring rule as Phase 2.
- **Phase 4: Route Insertion.** The algorithm then looks at rejected orders in decreasing revenue order and tries to insert each into an existing compatible route. Candidate cars are sorted by current distance to the pickup station and available time. Only positions near the chronological insertion point are checked. A simulated insertion is accepted only if

$$3R_k - 0.02 \max(0, \Delta M) \left(1 + \frac{3B_{\text{used}}}{B}\right) > 0$$

  and the total relocation budget remains feasible.
- **Phase 5: One-removal Swap.** If a valuable rejected order still cannot be inserted directly, the algorithm removes one lower-revenue accepted order from a compatible route and inserts the rejected order near its chronological position. Only capped candidate cars, removable orders, and insertion positions are checked. A swap is accepted when $3(R_{\text{new}} - R_{\text{removed}}) - 0.02 \max(0, \Delta M) \left(1 + \frac{3B_{\text{used}}}{B}\right) > 0$. This fixes cases where greedy assignment used a scarce time slot or car level for a lower-value order.
- **Phase 6: Last-order Replacement.** The final repair is even cheaper. For each high-revenue rejected order, a compatible car is rolled back to the state before its last accepted order. If the rejected order can replace that last order with positive revenue gain after the dynamic move penalty, the old order and its relocation record are cancelled.
- **Phase 7: Finalization.** Cancelled relocation records are removed and accepted revenue is computed. Among all variants, the algorithm keeps the feasible plan with the largest accepted revenue.

Each repaired route is simulated from a car's initial station and time 0 before replacing the old route. On very small instances, a bounded exact DFS is used as a final improvement; this is skipped on large hidden instances.

If Q is the number of six-hour buckets, building one look-ahead table costs $O(n_K + n_S n_L Q)$. The greedy assignment scan is $O(n_K C)$ for each parameter variant, where C is the average number of compatible cars inspected per order; in the worst case $C = n_C$, giving $O(n_K n_C)$. With V parameter variants, the overall complexity of Approach B is $O(V(n_K n_C + n_K + n_S n_L Q) + R)$, where R is the cost of bounded staging and route-repair searches. Since V and the repair limits are fixed constants, and large instances use only two variants, the practical scaling is $O(n_K n_C + n_S n_L Q)$, with the $n_K n_C$ car-scan term usually dominating on large instances. The pre-build stage runs both Approach A and Approach B and keeps the feasible solution with larger profit.

Optimization: batch local IP repair

The pre-build stage is fast, so we use the remaining time to improve the solution by exact optimization on small neighborhoods. First, the assignment is converted into routes for all cars. In each iteration, we sample a small batch of cars whose routes are relatively inefficient. For a car route c, define $v_c = \text{sales}_c/(1 + \text{move}_c)$. Routes with smaller v_c are more likely to be released. We then use a softmax-style sampling without replacement:

$$w_c = \exp\left(-\frac{v_c - v_{\min}}{\max(10^{-9},\ |\bar{v}| \cdot \theta)}\right)$$

where $\theta = 0.35$ is the temperature, $v_{\min}$ is the smallest route value, and $\bar{v}$ is the average route value among nonempty routes. A lower-value route therefore receives a larger sampling weight, but the temperature keeps the search from always selecting the same cars.

The default batch size is fixed at $b = 5$. The orders currently served by these cars are released, while routes of all other cars remain fixed. We then add a capped list of compatible candidate orders, where the candidate order limit is $m = 160$. Candidate orders are sorted with released orders first, then by decreasing revenue, pickup time, and order ID; currently rejected compatible orders are also eligible. This keeps the local IP small while still giving it a chance to recover from a bad earlier assignment.

For this batch, we solve a local network-flow IP. It uses start variables for a selected car's first order, link variables for consecutive orders on a selected car, and acceptance variables for candidate orders. The local constraints are the same logical constraints as in Problem 1: each accepted order has one incoming arc, each selected car starts at most one route, flow is conserved, and the moving time used by the repaired batch does not exceed the remaining budget. If the local IP returns a set of routes with higher total profit and feasible moving time, we replace the old routes; otherwise, the incumbent solution is kept. Since the incumbent is never overwritten by a worse or infeasible repair, the algorithm can stop at any time and still return a feasible plan.

The full procedure is:

Input: instance file and three-minute time limit.
1. $A^{(1)} \leftarrow$ Approach A construction.
2. $A^{(2)} \leftarrow$ Approach B construction.
3. $A \leftarrow \arg\max\{\text{profit}(A^{(1)}), \text{profit}(A^{(2)})\}$.
4. **while time remains do**
5. compute $v_c = \text{sales}_c/(1 + \text{move}_c)$ for each nonempty route;
6. sample up to $b = 5$ routes without replacement using softmax weights w_c;
7. release those routes and compute the remaining relocation budget;
8. build at most $m = 160$ compatible candidate orders;
9. solve the local IP with per-IP time limit $\tau = 2$ seconds;
10. if no feasible IP solution is found, retry with smaller candidate prefixes;
11. accept the repair only if profit improves;
12. **return** A and its relocation plan.

The smaller-prefix retry is also fixed in the implementation. The first local IP uses up to 160 candidates. If it returns no feasible solution, the candidate prefix is halved and retried, so one repair iteration tries at most four IPs with candidate limits 160, 80, 40, and 20. Each attempt receives at most $\tau = 2$ seconds and also respects the global wall-clock deadline.

Suppose a local batch has b cars and at most m candidate orders. The local IP contains $O(bm)$ start arcs and $O(bm^2)$ order-to-order arcs in the worst case, so it has $O(bm^2)$ binary variables and constraints. In our submitted configuration, $b = 5$ and $m \leq 160$, hence the local IP size is bounded by a constant: $O(bm^2) = O(5 \cdot 160^2)$.

The expensive part is therefore controlled directly by the wall-clock budget. With total remaining optimization time T, per-IP limit $\tau = 2$, and at most four IP attempts per repair iteration, the number of completed repair iterations is at most $O(T/(4\tau))$, and the number of IP solves is at most $O(T/\tau)$. The route-value computation, softmax sampling, and candidate sorting add only polynomial preprocessing per iteration, dominated in practice by scanning orders and sorting the capped candidate list. Since all major repair parameters are fixed constants, the optimization phase is best viewed as a time-budgeted constant-size exact repair loop whose runtime is bounded by the three-minute wall-clock limit.

Problem 4

To systematically evaluate the effectiveness and robustness of our proposed heuristic algorithm, we conducted a comprehensive numerical study following the factorial design methodology, generating random instances designed with different factors and scenarios.

General Settings: Across all instances, the network consists of $n_S = 100$ stations and $n_L = 10$ car levels. The planning horizon starts at `2023/01/01 00:00`, given the length sampled as $n_D \sim U(7, 100)$ days. Pickup times s_k

are rounded to 30-minute slots and fall within the first $n_D - 2$ days to ensure rentals finish within the horizon. Rental durations are uniformly distributed between 2 and 48 hours. Car levels are assigned as $q_c \sim U(1, 10)$. Hourly rate for a level-l car is strictly increasing: $F_1 \sim U(150, 250)$, with level increments $F_{l+1} - F_l \sim U(50, 150)$. Last, the moving time between stations is symmetric, where $t_{i,i} = 0$ and $t_{i,j} = 90$ minutes for all $i \neq j$.

We designed 14 scenarios driven by 5 operational factors to test the algorithm under various environments:

- **Factor A: System Load ($n_K : n_C$):** We vary the order-to-car ratio to test algorithm efficiency under different supply-demand pressures. The baseline is a 1:1 ratio ($n_C = 100, n_K = 100$). We introduce a *Low Load* (1:3 ratio, $n_C = 100, n_K = 33$), a *Medium Load* (3:1 ratio, $n_C = 80, n_K = 240$), and a *Highly Saturated* scenario (10:1 ratio, $n_C = 80, n_K = 800$).
- **Factor B: Level Mismatch:** Compared to the baseline uniform distribution($q_c \sim U(1, n_L = 10), \forall c \in C$), the 8:2 Skew scenario assigns 80% of customer requests to low-tier odd levels ($P(l_k = x) = \frac{1}{5}, x \in (1, 3, 5, 7, 9)$) while 80% of the level a car belongs to are even ($P(q_c = y) = \frac{1}{5}, y \in (2, 4, 6, 8, 10)$). This forces the algorithm to heavily utilize the upgrade mechanism.
- **Factor C: Spatial Flow Dynamics:**
  - **Odd/Even Flow:** We enforced all cars to be initialized at odd-numbered stations, while all orders depart from even and return to odd stations, making relocation mandatory.
  - **Hub Effect:** To simulate geographical resource accumulation (e.g., airports or city centers), requested pickup-and-return-station probabilities are heavily skewed. Normal stations have a weight of 1, while stations in a "Hub Group" (5 stations per group) have a weight of 100 when doing uniform sampling. We test 1-Hub and 2-Hub variants.
- **Factor D: Temporal Volatility:**
  - **Weekend Effect:** Pickup probabilities on Fridays, Saturdays, and Sundays are weighted 5 times higher than on other weekdays (which are uniformly distributed).
  - **Peak Bursts:** To simulate consecutive holiday effects, order pickups are drawn from normal distributions $N(\mu, \sigma^2)$ rather than uniformly. We use 1 or 3 temporal centers (μ) with a highly concentrated standard deviation of $\sigma = 180$ minutes. These bursts represent extreme demand to test the cleaning and readiness buffer bottleneck.
- **Factor E: Relocation Budget (B):** We scale B to observe behavior under relocation pressure. Given an unconstrained budget ($B = 10^6$ minutes) for pure objective testing, a moderate budget ($B = 300n_D$), a tight budget ($B = 30n_D$), and a zero-budget strict case ($B = 0$).

The parameter configurations for all 14 scenarios are summarized in Table 3. Our numerical study is divided into two stages: a breadth test (30 instances per scenario across all 14 scenarios) to verify general viability, and a depth robustness test (128 instances per scenario on 7 selected scenarios).

Table 3: Summary of the 14 experimental scenarios and their specific parameter configurations.

Scenario	Main factor	Ratio $n_K : n_C$	Levels dis.	Stations dis.	Time/Budget
S1	Baseline	1:1	random	random	random, $B = 10^6$
S2	Very high load	10:1	random	random	random, $B = 10^6$
S3	Low load	1:3	random	random	random, $B = 10^6$
S4	High load	3:1	random	random	random, $B = 10^6$
S5	Level mismatch	1:1	8:2 skew	random	random, $B = 10^6$
S6	Forced relocation	1:1	random	odd/even	random, $B = 300n_D$
S7	One hub	1:1	random	hub 1G	random, $B = 300n_D$
S8	Two hubs	1:1	random	hub 2G	random, $B = 300n_D$
S9	Weekend effect	1:1	random	random	weekend, $B = 10^6$
S10	One peak	1:1	random	random	one peak, $B = 10^6$
S11	Three peaks	1:1	random	random	three peaks, $B = 10^6$
S12	Zero relocation budget	1:1	random	random	random, $B = 0$
S13	Tight budget	1:1	random	random	random, $B = 30n_D$
S14	Moderate budget	1:1	random	random	random, $B = 300n_D$

To formulate the performance metrics, let A be the accepted reward of a solution and T be the total reward of all input orders. Then the project profit is profit $= A - 2(T - A) = 3A - 2T$.

For a fixed instance, T is constant, so maximizing profit is equivalent to maximizing $3A$. Whenever we compare a heuristic against the IP optimum in the tables and histograms, we use the accepted-reward optimality gap. For any method $m \in \{\text{base}, \text{heuristic}\}$,

$$\text{gap}_m = \frac{3A_{\text{IP}} - 3A_m}{3A_{\text{IP}}} \times 100\% = \frac{\text{profit}_{\text{IP}} - \text{profit}_m}{\text{profit}_{\text{IP}} + 2T} \times 100\%.$$

To evaluate our algorithm systematically, we compare it against both the exact IP optimum (upper bound) and a simple chronological heuristic (lower baseline) which first sorts all orders by pickup time and breaking ties by order ID: for each order, it assigns the feasible compatible car requiring the least immediate moving time chronologically; the order is directly rejected if no car is feasible. As the latter algorithm has no look-ahead or repair, our sophisticated algorithm should clearly beat it in theory.

The following metrics are reported in the subsequent tables:

- **Base gap / Our gap (%):** The accepted-reward optimality gap compared to the IP optimum, defined as

$$\frac{3A_{\mathrm{IP}} - 3A_m}{3A_{\mathrm{IP}}} \times 100\%, m \in \{\text{base}, \text{our heuristic}\}$$

- **Improve (%):** The relative accepted-reward improvement of our method over the baseline, computed as

$$\frac{3A_{\mathrm{our}} - 3A_{\mathrm{base}}}{3A_{\mathrm{base}}} \times 100\%$$

- **CR (%):** The order Completion Rate, representing the percentage of total orders successfully assigned.

To keep the tables readable, we report the 10-second version of our algorithm used for the later-discussed larger 128-instance robustness study shown below.

Table 4: All-scenario benchmark for the 10-second proposed method. Each row summarizes 30 generated instances. Gaps are measured against the exact IP (upper benchmark); improvement is measured against the simple baseline.

Scenario	Base gap (%) $\mu \pm \sigma$	Our gap (%) $\mu \pm \sigma$	Improve (%) $\mu \pm \sigma$	Base CR (%) avg.	Our CR (%) avg.
S1	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	99.7	99.7
S2	0.91 ± 2.02	0.26 ± 0.65	0.69 ± 1.52	98.0	97.9
S3	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	99.5	99.5
S4	0.19 ± 0.57	0.03 ± 0.19	0.16 ± 0.43	99.2	99.2
S5	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	99.7	99.7
S6	$\mathbf{3.43 \pm 10.15}$	0.00 ± 0.00	$\mathbf{5.16 \pm 15.87}$	95.8	95.8
S7	$\mathbf{2.62 \pm 7.09}$	0.00 ± 0.00	$\mathbf{3.33 \pm 9.15}$	96.7	97.2
S8	$\mathbf{5.07 \pm 11.30}$	0.05 ± 0.16	$\mathbf{7.32 \pm 17.99}$	93.9	94.8
S9	0.00 ± 0.00	0.00 ± 0.00	0.00 ± 0.00	99.9	99.9
S10	$\mathbf{7.79 \pm 3.60}$	0.48 ± 0.52	$\mathbf{8.08 \pm 4.26}$	89.7	89.8
S11	0.18 ± 0.74	0.00 ± 0.00	0.18 ± 0.77	99.9	99.9
S12	$\mathbf{4.64 \pm 6.01}$	1.04 ± 2.86	$\mathbf{4.16 \pm 7.04}$	15.6	15.6
S13	$\mathbf{38.61 \pm 7.52}$	0.84 ± 1.36	$\mathbf{63.86 \pm 20.32}$	38.2	42.7
S14	$\mathbf{4.20 \pm 10.81}$	0.07 ± 0.28	$\mathbf{6.21 \pm 17.61}$	94.4	95.7

By grouping the 14 scenarios within 5 factors, by comparing each to scenario S1, it is observed that

A. **S2-S4:** The higher the load of orders, it is more difficult for the heuristic algorithms to reach the optimum.

B. **S5:** Algorithms have little barrier utilizing the upgrading mechanism, given the gaps equal to 0.

C. **S6-S8 / S12-S14:** The most dramatic performance gains are seen here, giving the conclusion that the relocation of cars from station to station has significant impact to the algorithm. The most dramatic performance gains are observed in these scenarios. Specifically, S13 (Tight Budget) shows a massive 63.86% improvement over the baseline. This can be explained through two lenses:

   - **Budget Depletion:** In S13, the relocation budget is extremely scarce. Simple heuristics fail because they exhaust the budget on the first few orders they encounter. Once B is spent, the fleet remains "frozen," and high-value orders at remote stations must be rejected with heavy penalties.
   - **Strategic Selectivity:** Our algorithm's look-ahead scoring and Relocation-Value Gate treat moving time as a high-value asset. By calculating the ROI of every relocation ($\frac{3R_k}{M}$), we ensure that the limited budget is spent only on orders that maximize the net profit swing.

   Furthermore, in S8 (Two Hubs), our algorithm maintains an optimality gap of 0.05% compared to the baseline's 5.07%. This demonstrates that look-ahead scoring successfully identifies the marginal utility of relocation even in "car graveyard" scenarios, effectively quantifying the "shadow price" of moving time and rebalancing the fleet across the 100-day horizon.

D. **S9-S11:** The scenarios' result seems to be extreme, while S10 shows strong effect on the algorithm in time peaks, both S9 and S11 have weak significance so. It is inferred that the polarized results are due to the excessive frequency of the peaks within the fixed $n_D = 100$, resulting in the factor impact reduction.

In conclusion, the effectiveness of our designed algorithm is strongly supported following the ”Our Gap” column in Table 4: our gap is smaller than the base gap for all scenarios and is consistent to a certain extent (given a range variants about 1 percent only). To double check our conclusion, we then selected S6, S7, S8, S10, S12, S13, and S14 to generate more instances and plotted their gap histograms for deeper studies since they receive higher gaps ($\mu > 1$) between the upper(IP) and lower(simple) benchmark algorithm (base gap) in comparison.

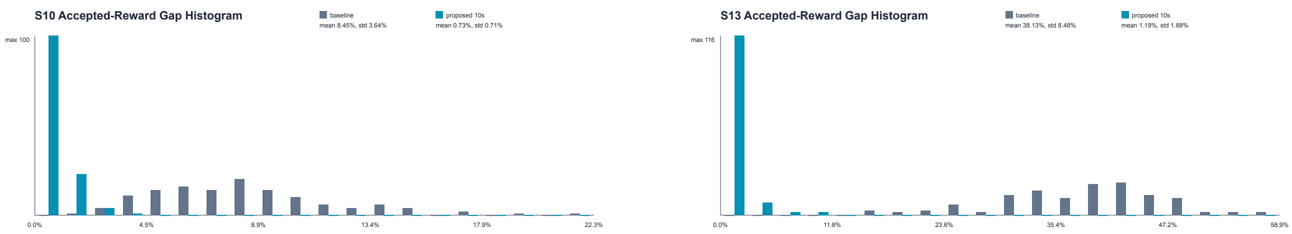
Deeper Study and Conclusion: Robustness Across 128 Instances

Table 5: Seven selected 128-instance scenario studies using the 10-second proposed method.

Scenario	Base gap (%) $\mu \pm \sigma$	Our gap (%) $\mu \pm \sigma$	Improve (%) $\mu \pm \sigma$	Base CR (%) avg.	Our CR (%) avg.
S_6	6.90 ± 13.72	0.00 ± 0.00	10.72 ± 22.87	90.5	90.5
S_7	3.44 ± 9.95	0.01 ± 0.08	5.12 ± 15.46	95.2	95.8
S_8	6.24 ± 12.09	0.03 ± 0.13	8.99 ± 18.68	91.7	92.8
S_{10}	8.45 ± 3.64	0.73 ± 0.71	8.60 ± 4.44	89.9	90.2
S_{12}	2.49 ± 4.38	0.36 ± 1.29	2.42 ± 5.65	15.7	15.7
S_{13}	38.13 ± 8.48	1.19 ± 1.88	62.63 ± 22.31	32.7	36.3
S_{14}	6.21 ± 12.65	0.07 ± 0.29	9.20 ± 20.10	91.7	93.1

By observation, the ”Our gap” column remains consistent when the instances grow even for the ”stressed” environments. As shown in Table 5, our proposed method maintains an average optimality gap of less than 1.3% across all $128 * 7 = 896$ test instances. Notably, in S_6 (Forced Relocation), our algorithm perfectly matched the IP optimum (0.00% gap) across all 128 instances, demonstrating that our relocation-aware look-ahead successfully pre-positions the fleet even under mandatory relocation constraints.

The experimental results strongly support our algorithm design. By combining a strategic look-ahead pre-build with targeted local IP repair, we achieve near-optimal solutions within the strict three-minute time limit.



S10: Demand surges

S13: Tight budget

Figure 1: Representative optimality-gap histograms highlighting distinct operational stressors: demand surges (S10) and tight budget bottlenecks (S13), reporting the mean and standard deviation of both algorithms.

The histograms in Figure 1 contrast our 10-second proposed method (teal) against the baseline (grey) for S10 & S13. These distributions provide clear evidence of our algorithm’s effectiveness regarding 2 key criteria:

1. **Near-Optimality:** In both S10 and S13, the teal distributions are heavily clustered at the leftmost edge of the x-axis. For S10, the vast majority of instances result in a gap between 0.0% and 0.5%. Even in the most constrained scenario, S13, the algorithm maintains a mean gap of only 1.19%. This extreme concentration near the 0% mark proves that our heuristic consistently identifies solutions that are practically indistinguishable from the exact IP optimum.

2. **Superiority over Benchmark:** The histograms reveal a dramatic ”gap shift” between the two methods.
   - **Reliability:** Our method shows significantly lower variance (a narrow teal spike), whereas the baseline is highly volatile (a wide grey spread).
   - **Risk Mitigation:** In S13, the baseline exhibits gaps reaching up to 58.9% because it exhausts the relocation budget prematurely. Our method successfully compresses the entire distribution toward the origin, ensuring that even our worst-case performance is vastly superior to the baseline’s average.

Based on the above analysis, it is demonstrated that our framework and all designs satisfy the project objectives, delivering a heuristic that provides high-profit, near-optimal plans while remaining resilient to the operational stressors of budget scarcity and demand surges. The full set of instances, optimal plans, run-time records, and benchmark rows is preserved in `scenario_study_128_results.tgz`, and the source code is publicly available on our GitHub repository (`https://github.com/089487/OR_Midterm_project`) for reproducibility.